

Plan de pruebas y evaluación

Propósito y niveles

Las pruebas comprueban contratos y comportamiento del software. La evaluación de recuperación comprueba los documentos obtenidos. La evaluación semántica comprueba contenido generado. Ninguna de estas capas sustituye a las otras. Las evidencias automáticas están en reports/tests.xml, reports/coverage.json y los JSON de evaluación; el resumen PDF se genera a partir de esos archivos.

Casos automatizados

- P01. Entrada vacía, corta, de espacios o extensa: rechazo Pydantic/422. RF01.
- P02. Matriz de impacto e interrupción: prioridad y horas de primera respuesta correctas. RF05.
- P03. Correos, RUT y secretos etiquetados: redacción antes de generación. RNF03.
- P04. Fragmentos de 30 palabras y solapamiento 8: límites y continuidad comprobables. RF02.
- P05. Corpus vacío, solapamiento inválido o ruta fuera de data: fallo explícito. RF02.
- P06. Consulta Firefox: recuperación de fuentes internas y externas. Filtro: solo el tipo solicitado. RF02.
- P07. Consulta de astronomía sin vocabulario pertinente: abstención y ninguna cita. RF06.
- P08. Presupuesto de contexto y k: no superar límites configurados. RNF05.
- P09. Id o extracto inventado, o ausencia de citas: rechazo y estado invalid_output. RF06.
- P10. Error de proveedor: provider_error, sin cambiar a demo. RF04.
- P11. Traza guardada: no contiene texto identificador del ticket ni correo. RF08.
- P12. Instrucción hostil del ticket queda como dato en user: comprobar separación de prompt. Es prueba estructural, no prueba de inmunidad del LLM.
- P13. Contrato Ollama chat y embeddings con mocks: request, esquema, uso y normalización. No realiza inferencia real.
- P14. Recuperación híbrida con vectores controlados: ambas señales participan. No mide embeddings descargados.
- P15. API e interfaz estática: solicitud completa, estado y origen externo. RF01/RF07.
- P16. Métricas multiclase, proveedor explícito y resultados semánticos nulos. RF09.
- P17. Conjuntos dev/test sin ids o textos duplicados. No demuestra independencia semántica ni representatividad.

Ejecutar: `.venv/Scripts/python.exe -m pytest --cov=catia --cov-report=json:reports/coverage.json --junitxml=reports/tests.xml`. La cobertura mide líneas ejecutadas y no calidad total ni porcentaje de la pauta. Los avisos de compatibilidad de dependencias deben conservarse en la evidencia si aparecen.

Dataset y protocolo

evaluation/dataset.json contiene 8 ejemplos dev y 20 test, todos sintéticos. Sus etiquetas provienen del diseño del implementador y deben revisarse por ambos estudiantes antes de una defensa formal. Las categorías y frases son sencillas: resultados altos pueden reflejar facilidad del corpus y no generalización. No se afirma que sea una muestra aleatoria o un test ciego independiente. Agregar tickets reales requiere autorización y anonimización apropiada.

Separar el uso: dev para elegir k, umbral y prompt; test para reportar una configuración congelada. No modificar las etiquetas o ejemplos difíciles para elevar una cifra. Si se cambia tras ver test, registrar el cambio y crear otro conjunto para validación final. Registrar corpus, prompt, modelo, configuración y fecha de cada ejecución.

Métricas definidas

Accuracy categoría: aciertos / casos. Macro-F1: media no ponderada de F1 por categoría, con zero_division=0. Matriz de confusión: filas verdaderas y columnas predichas en el orden labels del JSON. Accuracy de prioridad mide la regla con impacto declarado, no comprensión de urgencia por LLM.

Precisión de contexto documental: documentos relevantes recuperados / documentos recuperados distintos. Recall documental: relevantes recuperados / relevantes etiquetados. Se deduplican fragmentos del mismo documento; estas métricas no equivalen a precisión por chunk ni a Context Precision ponderada por ranking de RAGAS. Los casos sin documentos relevantes se excluyen del promedio de precisión/recall y se usan para abstención. MRR considera la posición del primer fragmento relevante en la lista original.

Abstention accuracy compara ausencia de evidencia esperada con status=insufficient_context. Un provider_error no cuenta como abstención correcta. Integridad de citas de borradores cuenta citas aceptadas por el validador entre drafts: es una propiedad del filtro y no una estimación de fidelidad de todo el sistema. Reportar también estados y cobertura de respuestas para evitar ocultar rechazos.

Latencia p50 y p95 se calcula por ticket dentro del proceso, después de crear el índice. Excluye arranque, descarga del modelo, renderizado y percepción del usuario. Latencia demo no se atribuye a Ollama.

Evaluación humana

Completar evaluation/revision_humana.csv después de ejecutar el LLM. Descomponer cada respuesta en afirmaciones comprobables. Faithfulness = afirmaciones apoyadas por el contexto / afirmaciones comprobables; si no hay afirmaciones, marcar no aplicable y evaluar abstención, no asignar 1 automáticamente. Relevancia: 1 no responde; 2 responde escasamente; 3 parcialmente útil; 4 responde con omisiones menores; 5 responde directamente y pide lo necesario. Revisar que cada recomendación esté apoyada por su cita, que no se inventen precios o devoluciones y que el resumen no agregue hechos.

Los dos integrantes revisan primero por separado y resuelven desacuerdos dejando notas. Evitar que el único generador se autoevalúe y tomar su juicio como verdad. Un LLM juez puede añadirse como complemento, identificando modelo, prompt y sesgos, no como sustituto de esta revisión.

Experimentos reproducibles

También se registra el experimento de optimización v2 → v3. Consultar `reports/OPTIMIZACION_PROMPT.md` y los JSON de cada versión. La clasificación del LLM v2 quedó bajo la meta inicial; se mejoró la taxonomía con base en sus fallos y se reporta la reevaluación sin borrar el resultado anterior.

E1: recuperar con $k=2$ y $k=4$ sobre dev, manteniendo umbral y corpus. E2: comparar v1 y v2 con el mismo LLM, datos y retriever. E3: ejecutar `--baseline` sin documentos y comparar afirmaciones sin soporte con el RAG. E4: probar caso fuera de dominio, instrucciones hostiles, conflicto entre guía externa y política, y ambigüedad. E5: activar embeddings y documentar evaluación separada; los resultados lexical existentes no validan ese modo.

El simulador no lee el prompt para decidir: comparar v1 y v2 en demo no demuestra optimización de prompts. El experimento sin contexto de demo tampoco mide ventaja del LLM. Solo usar inferencia real para esos argumentos.

Pruebas manuales de aceptación

MA01: abrir aplicación en escritorio y móvil; comprobar lectura, foco de teclado y ausencia de desbordamiento. MA02: caso Firefox muestra fuentes de ambos tipos. MA03: ticket comercial con URGENTE permanece Baja con impacto individual. MA04: marcar caída organizacional eleva prioridad Alta/1 hora. MA05: astronomía produce abstención. MA06: detener Ollama, analizar y observar fallo explícito. MA07: descargar JSON y cotejar ids con pantalla. MA08: revisar un borrador real contra fuente y completar ficha humana. Registrar fecha, navegador, resultado y captura; no marcar estas pruebas aprobadas sin realizarlas.

Criterio de salida

La evidencia `reports/llm_smoke.json` registra una inferencia real. Los archivos `browser_checks.json` y las capturas registran las comprobaciones de interfaz efectivamente realizadas en modo demo. El informe automático incorpora la evaluación completa del LLM cuando existe su JSON. Consultar el estado de cada ejecución; la revisión humana de contenido no se marca como completada por estos scripts.

Pruebas automáticas pasan, recuperación cumple objetivos declarados, proveedor real puede ejecutar una consulta, las métricas semánticas están revisadas, ambos integrantes explican limitaciones y el caso tiene aprobación docente. Si falta inferencia real o revisión humana, la entrega es un prototipo preparado con validación parcial, no evidencia completa del mejor logro.